

Traffic Offense Management System

Toolkit Documentation

PHP, MySQL/MariaDB, Apache, HTML, CSS, JavaScript

Document type	Assignment toolkit document and repository documentation
Project type	Local web application for managing traffic offense records
Primary stack	PHP with MySQL/MariaDB running on Apache
Recommended path	Repository root as Toolkit_Documentation.docx or exported alongside README.md
Audience	Lecturer, reviewer, developer, or student setting up the project locally

Purpose of this document

This documentation explains the system, the technology choices, the setup process, a minimal working example, AI-assisted development prompts, lessons learned, common errors, and reference resources. It is written as project documentation so it can be included with the files pushed to GitHub.

Prepared for project submission

Document Map

Use this document from top to bottom when reviewing or setting up the project. The main sections are:

- Overview of the chosen technology
- System overview and repository structure
- Setup instructions
- Minimal working example
- AI prompts used and learning reflections
- Common errors and fixes
- Reference resources
- GitHub submission checklist

1. Overview of the Chosen Technology

The Traffic Offense Management System is built with a classic PHP web stack. PHP handles server-side page rendering and application logic, MySQL or MariaDB stores system data, and Apache serves the application locally through a browser. The frontend uses HTML, CSS, Bootstrap-style classes, and JavaScript/jQuery-style behavior for layout, buttons, modals, and basic interactivity.

This stack is suitable for an assignment project because it demonstrates the main skills needed to build and run a database-backed web application: local server setup, routing, configuration, sessions, database connectivity, page templates, and troubleshooting.

LOG IN CREDENTIALS :

Username: admin

Password: admin123

Technology	Role in the project
PHP	Runs the backend logic, starts sessions, loads configuration, includes pages, and connects the application to shared classes.
MySQL / MariaDB	Stores application data such as users, traffic offenses, drivers, records, and system settings.
Apache	Serves the application through localhost and supports rewrite behavior using the .htaccess file.
HTML	Provides page structure for the public pages, error page, and included content.
CSS / Bootstrap-style classes	Controls layout, buttons, modals, responsive spacing, and visual styling.
JavaScript / jQuery-style behavior	Supports modal actions, confirmation dialogs, and client-side page behavior.
Git and GitHub	Version control and repository submission for the assignment.

Development approach

The project was developed through rapid AI-assisted prototyping. The core behavior was described, generated with Codex, tested locally, debugged from runtime errors, and refined into a working PHP/MySQL structure. This approach helped move quickly while still requiring manual verification and understanding of each file.

2. System Overview

The system is designed to support the recording and management of traffic offense information. It provides a public landing page and an admin entry point. In a complete repository, the admin area is expected to handle controlled access to traffic offense records, driver or offender information, user administration, and reports.

2.1 Main Purpose

- Provide a browser-based system for traffic offense record management.
- Load shared system settings such as the application name and base URL.
- Use a database connection layer for application data.
- Provide a public homepage with an admin login link.
- Show a custom 404 page when a requested page is not available.
- Keep page layout consistent through shared header and footer includes.

2.2 Target Users

User type	Expected use
Administrator	Manages system users, settings, and protected records through the admin area.
Traffic office staff	Records and reviews traffic offense information.
Reviewer or lecturer	Runs the project locally and checks whether the required assignment toolkit is complete.
Developer/student	Maintains the code, configures the database, and improves the project.

2.3 High-Level Workflow

1. A browser request is sent to the local server, for example `http://localhost/traffic_offense/`.
2. `index.php` loads the project configuration from `config.php`.
3. `config.php` loads `initialize.php` and shared classes such as `DBConnection` and `SystemSettings`.
4. The application determines which page to load using the `p` query parameter. If no page is provided, home is loaded.
5. If the requested page exists, it is included. If it does not exist, the custom 404 page is shown.
6. The page is wrapped with shared header and footer files from the `inc` folder.
7. The homepage displays the configured system name and a login button that points to the admin area.

```
Browser request
-> index.php
    -> config.php
        -> initialize.php
        -> classes/DBConnection.php
        -> classes/SystemSettings.php
    -> inc/header.php
    -> requested page such as home.php
    -> inc/footer.php
```

3. Repository Structure

The uploaded repository files show the public entry point, configuration files, static pages, and license. The complete system also references additional folders that must be available when running the full application.

File or folder	Purpose
README.md	Short project summary. It identifies the project as a Traffic Offense Management System.
.htaccess	Apache configuration file used for rewrite and request handling behavior.
404.html	Custom page shown when a requested page cannot be found.
about.html	Public About content displayed inside the homepage section.
config.php	Bootstraps the app, starts sessions, loads initialize.php, creates the database connection, and defines helper functions.
home.php	Public landing page. It displays the system name and includes the About content.
index.php	Front controller. It decides which page to include based on the p query parameter.
initialize.php	Defines base URL, base path, database constants, and development data.
LICENSE	Project license file.
classes/DBConnection.php	Required class file for database connection. Referenced by config.php.
classes/SystemSettings.php	Required class file for system settings such as the displayed system name. Referenced by config.php.
inc/header.php and inc/footer.php	Required layout include files used by index.php.
admin/	Expected protected admin area linked from the homepage.
dist/img/no-image-available.png	Fallback image used by the validate_image helper function.

Important repository note

The uploaded files reference classes, includes, an admin folder, assets, and a database schema that must be present in the full repository. If those dependencies are missing, the app may load partially or fail with include, class, or database errors.

4. Setup Instructions

These instructions explain how to run the project locally using XAMPP. WAMP, MAMP, Laragon, or a manual Apache/PHP/MySQL installation can also be used with the same configuration principles.

4.1 Prerequisites

- XAMPP, WAMP, MAMP, Laragon, or another Apache/PHP/MySQL environment
- PHP 7.x or later
- MySQL
- phpMyAdmin or another database client
- A web browser such as Chrome, Edge, or Firefox
- A code editor such as Visual Studio Code
- Git, if the project is cloned from GitHub

4.2 Clone or Download the Repository

```
git clone <repository-url>
```

Alternatively, download the repository as a ZIP file from GitHub and extract it.

4.3 Move the Project into the Web Server Folder

For XAMPP on Windows, place the folder in:

```
C:\xampp\htdocs\traffic_offense
```

For Linux Apache installations, a common location is:

```
/var/www/html/traffic_offense
```

4.4 Start Apache and MySQL

1. Open the XAMPP Control Panel.
2. Start Apache.
3. Start MySQL.
4. Open the project in a browser at http://localhost/traffic_offense/.

4.5 Create and Import the Database

The project expects a database named `traffic_offense_db`. Create it in phpMyAdmin or run:

```
CREATE DATABASE traffic_offense_db;
```

If the full repository includes a database dump such as `traffic_offense_db.sql`, import it into this database using phpMyAdmin.

4.6 Configure the Database Connection

Database constants are defined in `initialize.php`. For a local XAMPP setup, the default values usually look like this:

```
define('DB_SERVER', 'localhost');  
define('DB_USERNAME', 'root');  
define('DB_PASSWORD', '');  
define('DB_NAME', 'traffic_offense_db');
```

Security note

Do not commit real production passwords, private credentials, or server secrets into a public GitHub repository. For a classroom localhost project, default local values may be acceptable, but production values should be stored securely outside the repository.

4.7 Configure the Base URL

The project uses a base URL constant. If the folder name changes, update the URL to match the local folder path.

```
define('base_url', 'http://localhost/traffic_offense/');
```

4.8 Run the Application

After starting Apache and MySQL, open:

```
http://localhost/traffic_offense/
```

The homepage should load and display the system name with a Login button. The Login button should route to:

```
http://localhost/traffic_offense/admin
```

5. Minimal Working Example

This section gives small examples that prove the core toolkit is working before the full application is tested.

5.1 PHP Hello World

Create hello.php inside the project folder:

```
<?php
echo 'Hello World from PHP!';
?>
```

Open the file in the browser:

```
http://localhost/traffic_offense/hello.php
```

Expected result: the browser displays Hello World from PHP.

5.2 Quick Database Connection Test

Create db_test.php inside the project folder to confirm that PHP can reach MySQL/MariaDB:

```
<?php
$conn = new mysqli('localhost', 'root', '', 'traffic_offense_db');

if ($conn->connect_error) {
    die('Connection failed: ' . $conn->connect_error);
}

echo 'Database connected successfully.';
?>
```

Open:

```
http://localhost/traffic_offense/db_test.php
```

Expected result: Database connected successfully.

5.3 Project Route Test

The main route should load the homepage:

```
http://localhost/traffic_offense/
```

A missing page should load the custom 404 page:

```
http://localhost/traffic_offense/?p=missing_page
```

6. AI Prompts Used and Learning Reflections

This project used AI as a coding assistant during rapid prototyping, debugging, testing, and documentation. The workflow followed a vibe-coding approach: describe the intended behavior, let Codex generate or revise code, run the project locally, read the errors, ask AI to debug the error, apply the fix, and test again.

6.1 Sample Prompts Used

Stage	Prompt used
Project generation	Assume the role of a vibe coder working in Codex. Generate a PHP and MySQL Traffic Offense Management System with a public landing page, admin entry point, database configuration, reusable header/footer includes, and simple front-controller routing.
File structure refinement	Refactor the PHP project so index.php controls page loading, config.php bootstraps sessions and shared settings, initialize.php stores constants, and home.php displays the landing page.
Database setup	Create the database connection flow for a local XAMPP environment using MySQL/MariaDB. Use a database named traffic_offense_db and make the connection reusable across the project.
Debugging	Review this PHP error and trace the likely cause through index.php, config.php, initialize.php, DBConnection.php, and SystemSettings.php. Explain the fix and show the corrected code.
Routing test	Test the routing logic conceptually. What should happen when p is missing, when p=home, when a page exists, when a folder exists, and when the requested page does not exist?
UI testing	Act as a QA tester for this PHP web app. Create a manual test checklist for homepage load, admin login link, missing page handling, database connection, and mobile browser behavior.
Documentation	Convert the project into clean assignment toolkit documentation. Include technology overview, setup instructions, minimal working example, AI prompts used, learning reflections, common errors, and official reference resources.

6.2 Debugging and Testing with AI

AI was used as a debugging partner rather than as a replacement for running the code. The practical workflow was:

1. Generate the first version of the PHP structure in Codex.
2. Run the system through localhost using Apache and MySQL.
3. Capture PHP errors, include errors, routing errors, and database connection errors.

4. Ask AI to explain the error source and propose a fix.
5. Apply the fix manually in the code editor.
6. Retest the exact route or page that failed.
7. Repeat until the homepage, routing, 404 page, and database connection behaved as expected.

6.3 Learning Reflections

- The project made the PHP request lifecycle clearer: browser request, entry file, configuration, included page, and rendered output.
- Using Codex helped speed up scaffolding, but every generated file still needed review and local testing.
- Debugging include paths taught the importance of folder structure and correct relative paths in PHP projects.
- Database errors showed that a project can have correct PHP syntax but still fail if the database name, credentials, or schema are missing.
- The front-controller pattern in index.php helped keep page loading centralized, but it also showed why route validation and security checks matter.
- The documentation process helped clarify the system for someone who did not write the code.
- AI-assisted development works best when prompts are specific, errors are copied accurately, and the final code is tested instead of assumed to work.

7. Common Errors and How to Resolve Them

Error or symptom	Likely cause	Resolution
Blank white page	PHP error display may be off, or a fatal PHP error occurred.	Check Apache/PHP error logs. Temporarily enable error reporting during local development. Review recent file changes.
Failed to open stream: No such file or directory	A required file such as initialize.php, DBConnection.php, header.php, or footer.php is missing or in the wrong path.	Confirm the complete folder structure exists and that include paths match the actual file locations.
Class DBConnection not found	classes/DBConnection.php is missing, not loaded, or contains a class name mismatch.	Check that the file exists, is included by config.php, and defines the expected DBConnection class.
Database connection failed	MySQL is not running, the database does not exist, or credentials are incorrect.	Start MySQL, create traffic_offense_db, update DB constants, and test with db_test.php.
Unknown database traffic_offense_db	The expected database has not been created or imported.	Create the database and import the SQL dump if one is provided.
404 page appears unexpectedly	The requested p parameter does not match an existing PHP file or folder.	Check the URL and confirm the target page exists in the repository.
Login button opens a missing page	The admin folder is missing or Apache rewrite rules are not configured correctly.	Confirm the admin directory exists and that Apache can access the project folder.

Error or symptom	Likely cause	Resolution
.htaccess rules do not work	Apache rewrite module may be disabled or AllowOverride is not enabled.	Enable mod_rewrite and allow .htaccess overrides in Apache configuration.
Images not showing	Asset file path is wrong or the fallback image is missing.	Confirm the dist/img folder exists and contains no-image-available.png.
Changes do not appear in browser	Browser cache or server cache is showing an old version.	Hard refresh the page, clear cache, or restart Apache during local testing.

8. Security and Quality Notes

Because this is an assignment project, the priority is to demonstrate the working toolkit and understand the system. However, the following improvements should be considered before any real deployment:

- Do not store real passwords or production credentials directly in initialize.php.
- Avoid committing private admin credentials to a public repository.
- Validate the p parameter before including files to reduce the risk of unsafe dynamic inclusion.
- Use password_hash and password_verify for real user passwords instead of storing plain or weak hashes.
- Restrict access to admin pages using proper session checks.
- Use prepared statements for database queries to reduce SQL injection risk.
- Keep PHP, Apache, MySQL/MariaDB, and dependencies updated.
- Add a database schema or SQL dump to the repository so reviewers can run the full system.

9. Manual Test Checklist

Test case	Steps	Expected result
Homepage loads	Visit http://localhost/traffic_offense/ .	The public landing page appears with the system name and login button.
About content loads	Scroll through the homepage.	The About content is displayed inside the page section.
Admin link opens	Click the Login button.	Browser navigates to the admin area or shows the admin page if the folder exists.
Missing page handling	Visit ?p=missing_page .	The custom 404 page is displayed.
Database connection	Run db_test.php or load a page that depends on the database.	The database connects without credential or schema errors.
Mobile detection helper	Test from a mobile browser or device emulator.	Mobile user agents are detected by the helper function when used.
Assets load	Open the homepage and inspect styling/images.	CSS, JavaScript, and image assets load without broken paths.

10. Reference Resources

The following resources were used as reference material for the technology stack and setup process:

- [PHP Manual - Getting Started](https://www.php.net/manual/en/getting-started.php) - <https://www.php.net/manual/en/getting-started.php>
- [MySQL - Getting Started with MySQL](https://dev.mysql.com/doc/mysql-getting-started/en/) - <https://dev.mysql.com/doc/mysql-getting-started/en/>
- [Apache HTTP Server - .htaccess files](https://httpd.apache.org/docs/current/howto/htaccess.html) - <https://httpd.apache.org/docs/current/howto/htaccess.html>
- [XAMPP for Windows FAQ](https://www.apachefriends.org/faq_windows.html) - https://www.apachefriends.org/faq_windows.html
- [XAMPP for Linux FAQ](https://www.apachefriends.org/faq_linux.html) - https://www.apachefriends.org/faq_linux.html
- [Bootstrap 5.3 Introduction](https://getbootstrap.com/docs/5.3/getting-started/introduction/) - <https://getbootstrap.com/docs/5.3/getting-started/introduction/>
- [jQuery API Documentation](https://api.jquery.com/) - <https://api.jquery.com/>

11. GitHub Submission Checklist

Before pushing the final repository, confirm the following items are present:

- README.md with a short project description.
- Toolkit documentation file included in the repository.
- All required PHP files and folders included.
- Database SQL dump included if the system requires imported tables.
- No real production credentials committed to the repository.
- Local setup instructions tested from a fresh checkout.
- Homepage route and 404 route tested.
- Admin route verified or documented as requiring the admin folder.

Recommended Git commands:

```
git add .  
git commit -m "Add project toolkit documentation"  
git push origin main
```